Machine Learning – Practical Work No. 1

Linear Regression & Logistic Regression – Full Solution

العمل التطبيقي رقم 1 – التعلم الآلي: الانحدار الخطي والانحدار اللوجستي (حل كامل)

Step-by-step solution with explanations in English and Arabic | والعربية بالإنجليزية شرح مع بخطوة خطوة حل

> **Note:** Run each code block in its own notebook cell, in order, inside TP1_Regression.ipynb. Numeric results (MSE, R², accuracy) depend on the real datasets, so fill in your own values after running.

> **ملاحظة:** نفّذ كل مقطع كود في خلية مستقلة بالترتيب داخل ملف TP1_Regression.ipynb. القيم الرقمية (MSE, R², accuracy) تعتمد على البيانات الحقيقية، لذلك اكتب نتائجك أنت بعد التشغيل.

1. Setting Up the Environment

1. إعداد بيئة العمل

Step 1.1 – Install the libraries | الخطوة 1.1 – تثبيت المكتبات

EN Python ≥ 3.10 is required. Install all libraries with one pip command (or just open Google Colab, where they are pre-installed).

AR يجب أن يكون إصدار بايثون 3.10 أو أحدث. ثبّت كل المكتبات بأمر pip واحد (أو استخدم Google Colab حيث تكون المكتبات مثبتة مسبقاً).

```
pip install numpy pandas matplotlib seaborn scikit-learn jupyter
jupyter notebook          # then create a notebook named TP1_Regression.ipynb
```

Step 1.2 – Test the imports | الخطوة 1.2 – اختبار الاستيراد

EN Run this first cell. If no error appears, the environment is valid. If an import fails, run pip install <library_name>.

AR شغّل هذه الخلية أولاً. إذا لم يظهر أي خطأ فالبيئة سليمة. وإذا فشل استيراد مكتبة ما فثبّتها بالأمر pip install <اسم المكتبة>.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import sklearn

print("numpy        :", np.__version__)
print("pandas       :", pd.__version__)
print("scikit-learn :", sklearn.__version__)
```

2. Linear Regression: Salary vs Experience

2. الانحدار الخطي: التنبؤ بالراتب من سنوات الخبرة

EN Goal: fit the line Salary = intercept + slope × YearsExperience. Download Salary_Data.csv from Kaggle and put it next to the notebook.

AR الهدف: إيجاد المستقيم Salary = intercept + slope × YearsExperience. حمّل ملف Salary_Data.csv من Kaggle وضعه في نفس مجلد الدفتر (notebook).

Step 2a – Load and explore the data | الخطوة 2a – تحميل البيانات واستكشافها

EN pd.read_csv loads the file; head() shows the first rows, info() shows types and missing values, describe() gives mean, std, min, max and quartiles.

AR الدالة pd.read_csv تقرأ الملف؛ head() تعرض أول الصفوف، info() تعرض أنواع الأعمدة والقيم الناقصة، و describe() تعطي المتوسط والانحراف المعياري والقيم الدنيا والعليا والربيعات.

```
df = pd.read_csv("Salary_Data.csv")
print(df.head())
df.info()
print(df.describe())
```

Step 2b – Visualize and measure correlation | الخطوة 2b – الرسم وحساب الارتباط

EN A scatter plot shows whether the points follow a straight line. The Pearson coefficient r is between −1 and 1; a value close to +1 means a strong positive linear relationship.

AR مخطط التبعثر يوضح هل النقاط تتبع خطاً مستقيماً. معامل بيرسون r بين -1 و 1؛ وكلما اقتربت قيمته من +1 كانت العلاقة الخطية الموجبة قوية.

```
plt.figure(figsize=(7, 4))
sns.scatterplot(data=df, x="YearsExperience", y="Salary")
plt.title("Salary vs Years of Experience")
plt.grid(alpha=0.3)
plt.show()

print(df.corr())          # Pearson correlation matrix
```

EN Comment: the points are almost aligned and r is very close to 1 (about 0.97–0.98 for this dataset), so a linear model is appropriate.

AR التعليق: النقاط شبه مصطفة على خط مستقيم وقيمة r قريبة جداً من 1 (حوالي 0.97–0.98 في هذه البيانات)، إذن النموذج الخطي مناسب.

Step 2c – Train/test split (70/30) | الخطوة 2c – تقسيم البيانات إلى تدريب واختبار

EN We train on 70% and keep 30% unseen for honest evaluation. random_state=42 makes the split reproducible. Standardization is optional here: with a single feature, ordinary least squares gives the same predictions with or without scaling.

AR ندرّب النموذج على 70% ونحتفظ بـ 30% لم يرها النموذج لتقييم عادل. random_state=42 يجعل التقسيم قابلاً للتكرار. التوحيد (Standardization) اختياري هنا: مع متغير واحد تعطي المربعات الصغرى العادية نفس التنبؤات سواء وُحّدت البيانات أم لا.

```
from sklearn.model_selection import train_test_split

X = df[["YearsExperience"]]      # 2D (DataFrame) as required by scikit-learn
y = df["Salary"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, random_state=42)
print(X_train.shape, X_test.shape)
```

Step 2d – Train the model and interpret | الخطوة 2d – تدريب النموذج وتفسير المعاملات

```
from sklearn.linear_model import LinearRegression

lin = LinearRegression()
lin.fit(X_train, y_train)
```

```
print("Intercept (b0):", lin.intercept_)
print("Slope (b1)    :", lin.coef_[0])
```

EN Interpretation: the slope is the average salary increase for each additional year of experience (in salary currency units per year, about 9,000–10,000 in the usual version of the dataset). The intercept is the estimated salary of a person with 0 years of experience (starting salary).

AR التفسير: الميل (slope) هو متوسط الزيادة في الراتب عن كل سنة خبرة إضافية (بوحدة عملة الراتب لكل سنة، وهو في حدود 9000–10000 في النسخة المعتادة من البيانات). أما التقاطع (intercept) فهو الراتب المقدَّر لشخص بدون خبرة (أي راتب البداية).

Step 2e – Predict on the test set | الخطوة 2e – التنبؤ على مجموعة الاختبار

```
y_pred = lin.predict(X_test)
print(pd.DataFrame({"Actual": y_test.values, "Predicted": y_pred.round(0)}))
```

Step 2f – Plot the regression line | الخطوة 2f – رسم خط الانحدار

EN We draw the fitted line over the training points and over the test points to see how well it generalizes.

AR نرسم خط الانحدار فوق نقاط التدريب ونقاط الاختبار لنرى مدى قدرة النموذج على التعميم.

```
x_line = np.linspace(X.min(), X.max(), 100).reshape(-1, 1)
fig, ax = plt.subplots(1, 2, figsize=(12, 4), sharey=True)

for a, Xs, ys, title, col in [(ax[0], X_train, y_train, "Training set", "tab:blue"),
                              (ax[1], X_test,  y_test,  "Test set",     "tab:green")]:
    a.scatter(Xs, ys, color=col, label="Data")
    a.plot(x_line, lin.predict(pd.DataFrame(x_line, columns=["YearsExperience"])),
           color="red", label="Regression line")
    a.set_title(title); a.set_xlabel("Years of experience"); a.legend();
a.grid(alpha=0.3)
ax[0].set_ylabel("Salary")
plt.show()
```

Step 2g – MSE and R² | الخطوة 2g – حساب MSE و R²

EN MSE = average of squared errors (lower is better). R² = share of the salary variance explained by the model (closer to 1 is better).

AR MSE هو متوسط مربعات الأخطاء (كلما قلّ كان أفضل). أما R² فهو نسبة تباين الرواتب التي يفسرها النموذج (كلما اقترب من 1 كان أفضل).

```
from sklearn.metrics import mean_squared_error, r2_score

mse_lin = mean_squared_error(y_test, y_pred)
r2_lin  = r2_score(y_test, y_pred)
print(f"MSE = {mse_lin:,.2f}")
print(f"R2  = {r2_lin:.4f}")
```

Step 2h – Ridge, Lasso and Elastic Net | الخطوة 2h – التنظيم: Ridge و Lasso و ElasticNet

EN Ridge adds an L2 penalty (shrinks coefficients), Lasso adds an L1 penalty (can set coefficients exactly to 0), ElasticNet mixes both. Larger alpha means stronger regularization. The loop below fills the summary table automatically.

AR Ridge يضيف عقوبة L2 (تقلّص المعاملات)، و Lasso يضيف عقوبة L1 (قد تجعل بعض المعاملات صفراً تماماً)، و ElasticNet يمزج بينهما. كلما كبرت قيمة alpha زادت قوة التنظيم. الحلقة التالية تملأ جدول الملخص تلقائياً.

```
from sklearn.linear_model import Ridge, Lasso, ElasticNet
```

```
rows = [["LinearRegression", "-", mse_lin, r2_lin, lin.coef_[0]]]
for name, Model in [("Ridge (L2)", Ridge), ("Lasso (L1)", Lasso), ("ElasticNet",
ElasticNet)]:
    for alpha in [0.1, 1, 10]:
        m = Model(alpha=alpha, max_iter=100000).fit(X_train, y_train)
        p = m.predict(X_test)
        rows.append([name, alpha, mean_squared_error(y_test, p),
                     r2_score(y_test, p), m.coef_[0]])

results = pd.DataFrame(rows, columns=["Model", "alpha", "MSE (test)",
                                      "R2 (test)", "Estimated coefficient"])
print(results.to_string(index=False))
```

EN How to read the table: because there is only one feature and the salaries are large numbers, alpha = 0.1, 1 and 10 barely change the slope, so the four models perform almost the same. The effect would be visible with many correlated features or a very large alpha (for example 1000+). Lasso/ElasticNet with a huge alpha would push the coefficient towards 0 and R^2 would drop.

AR كيفية قراءة الجدول: بما أنه يوجد متغير واحد فقط والرواتب أعداد كبيرة، فإن القيم alpha = 0.1 و 1 و 10 بالكاد تغيّر الميل، ولذلك تكون أداءات النماذج الأربعة متقاربة جداً. يظهر الأثر بوضوح عند وجود متغيرات كثيرة مترابطة أو عند alpha كبيرة جداً (مثل 1000 فأكثر)، حيث يدفع Lasso/ElasticNet المعامل نحو الصفر ويهبط R^2.

Model	alpha	MSE (test)	R^2 (test)	Coefficient
LinearRegression	/	.....	.....	.....
Ridge (L2)	0.1 / 1 / 10	.....	.....	.....
Lasso (L1)	0.1 / 1 / 10	.....	.....	.....
ElasticNet	0.1 / 1 / 10	.....	.....	.....

EN (The table above is the template: copy your results printed by the code into it.)

AR (الجدول أعلاه هو النموذج المطلوب: انقل إليه النتائج التي تطبعها الشيفرة.)

3. Logistic Regression from Scratch: Heart Disease

3. الانحدار اللوجستي من الصفر: أمراض القلب

EN Here we use only NumPy for the model. Idea: compute z = Xw + b, convert it to a probability with the sigmoid σ(z) = 1/(1+e^−z), measure the error with cross-entropy, and update w and b with gradient descent.

AR هنا نستخدم NumPy فقط لبناء النموذج. الفكرة: نحسب z = Xw + b، ثم نحولها إلى احتمال بدالة السيغمويد σ(z) = 1/(1+e^−z)، ونقيس الخطأ بالإنتروبيا المتقاطعة (cross-entropy)، ثم نحدّث w و b بالانحدار التدريجي (gradient descent).

Step 3a – Load and prepare the data | الخطوة 3a – تحميل البيانات وتحضيرها

EN We load from OpenML, check missing values (fill numeric ones with the median), convert the target to integers 0/1, encode any categorical columns, scale features to [0,1] with MinMaxScaler, then split 70/30. The scaler is fitted on the training set only, to avoid data leakage.

AR نحمّل البيانات من OpenML ونفحص القيم الناقصة (نملأ العددية بالوسيط)، ونحوّل الهدف إلى أعداد صحيحة 1/0، ونرمّز الأعمدة الفئوية إن وُجدت، ثم نقيس الميزات إلى المجال [0,1] بـ MinMaxScaler، ثم نقسم 70/30. نطبّق الـ scaler على بيانات التدريب فقط لتفادي تسرب البيانات (data leakage).

```
from sklearn.datasets import fetch_openml
from sklearn.preprocessing import MinMaxScaler
```

```
try:
    data = fetch_openml(name='heart-disease', version=1, as_frame=True)
except Exception as e:
    print("heart-disease not available, falling back to heart-statlog:", e)
    data = fetch_openml(name='heart-statlog', version=1, as_frame=True)

df_h = data.frame.copy()
print(df_h.head())
print(df_h.isnull().sum())

# ---- target and features ----
target_col = data.target.name if hasattr(data.target, "name") else "target"
y_raw = df_h[target_col]
X_raw = df_h.drop(columns=[target_col])

# binary target -> integers 0/1 (the last sorted class = "disease present")
classes = sorted(y_raw.astype(str).unique())
y_all = (y_raw.astype(str) == classes[-1]).astype(int).values
print("classes:", classes, "-> 1 means", classes[-1])

# categorical columns -> numbers, missing values -> median
X_raw = pd.get_dummies(X_raw, drop_first=True).astype(float)
X_raw = X_raw.fillna(X_raw.median())
feature_names = X_raw.columns.tolist()

# ---- split, then scale (fit on train only) ----
Xtr, Xte, ytr, yte = train_test_split(X_raw.values, y_all, test_size=0.30,
random_state=42)
scaler = MinMaxScaler()
X_train = scaler.fit_transform(Xtr)
X_test  = scaler.transform(Xte)
y_train = ytr.reshape(-1, 1)      # column vectors (m, 1)
y_test  = yte.reshape(-1, 1)
print(X_train.shape, X_test.shape)
```

Note: If your instructor's notebook loads the data differently, keep their loading cell and only reuse the variable names X_train, X_test, y_train, y_test (y as column vectors).

ملاحظة: إذا كان دفتر الأستاذ يحمّل البيانات بطريقة مختلفة فاحتفظ بخلية التحميل الخاصة به وأعد فقط استخدام أسماء المتغيرات X_train, X_test, y_train, y_test (مع y كمتجهات عمودية).

Step 3b – Implement the model | الخطوة 3b – برمجة النموذج

EN Build the five functions in this order. Math: $a = \sigma(Xw+b)$; $J = -(1/m)\,\Sigma\,[\, y\cdot\log(a) + (1-y)\cdot\log(1-a)\,]$; $dw = (1/m)\,X^T(a-y)$; $db = (1/m)\,\Sigma(a-y)$. A tiny epsilon avoids log(0).

AR ابنِ الدوال الخمس بهذا الترتيب. الرياضيات: $a = \sigma(Xw+b)$؛ دالة الكلفة $J = -(1/m)\,\Sigma\,[\, y\cdot\log(a) + (1-y)\cdot\log(1-a)\,]$؛ المشتقة $dw = (1/m)\,X^T(a-y)$ و $db = (1/m)\,\Sigma(a-y)$. نضيف epsilon صغيراً لتفادي log(0).

```
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def initialize_parameters(n):
    w = np.zeros((n, 1))
    b = 0.0
    return w, b

def compute_cost_and_gradients(X, y, w, b, lambda_=0.0):
    """Cross-entropy cost + gradients (lambda_ ignored: no regularization)."""
    m = X.shape[0]
    a = sigmoid(X @ w + b)                    # (m, 1) probabilities
```

```
    eps = 1e-15
    cost = -(1/m) * np.sum(y*np.log(a + eps) + (1 - y)*np.log(1 - a + eps))
    dz = a - y
    dw = (1/m) * (X.T @ dz)                    # (n, 1)
    db = (1/m) * np.sum(dz)                    # scalar
    return cost, dw, db

def gradient_descent(X, y, w, b, alpha=0.1, num_iters=1000,
                     compute_fn=None, lambda_=0.0):
    if compute_fn is None:
        compute_fn = compute_cost_and_gradients
    costs = []
    for i in range(num_iters):
        cost, dw, db = compute_fn(X, y, w, b, lambda_)
        w = w - alpha * dw                     # minus sign: go downhill
        b = b - alpha * db
        costs.append(float(cost))
    return w, b, costs

def predict(X, w, b, threshold=0.5):
    probs = sigmoid(X @ w + b)
    return (probs >= threshold).astype(int)
```

Step 3c – Train without regularization and evaluate | الخطوة 3c – التدريب بدون تنظيم والتقييم

```
from sklearn.metrics import accuracy_score, confusion_matrix

w, b = initialize_parameters(X_train.shape[1])
w0, b0, costs0 = gradient_descent(X_train, y_train, w, b, alpha=0.1, num_iters=1000)

plt.plot(costs0)
plt.xlabel("Iteration"); plt.ylabel("Cost J"); plt.title("Convergence (no
regularization)")
plt.grid(alpha=0.3); plt.show()
print("Monotonic decrease:", all(np.diff(costs0) <= 1e-12))

y_pred0 = predict(X_test, w0, b0)
print("Accuracy:", accuracy_score(y_test, y_pred0))
cm0 = confusion_matrix(y_test, y_pred0)
print(cm0)
sns.heatmap(cm0, annot=True, fmt="d", cmap="Blues",
            xticklabels=["Pred 0", "Pred 1"], yticklabels=["True 0", "True 1"])
plt.show()
tn, fp, fn, tp = cm0.ravel()
print(f"TN={tn}  FP={fp}  FN={fn}  TP={tp}")
```

EN The cost starts at ln 2 ≈ 0.693 (all weights are zero, so every probability is 0.5) and must decrease at each iteration. If it increases or stays flat, check the sigmoid and the sign in w = w − α·dw.

AR تبدأ الكلفة من ln 2 ≈ 0.693 (لأن كل الأوزان أصفار فكل الاحتمالات 0.5) ويجب أن تنخفض في كل تكرار. إذا ارتفعت أو ثبتت فتحقق من دالة السيغمويد ومن إشارة التحديث w = w − α·dw.

Cell	Meaning (medical context)	المعنى (في السياق الطبي)
TN (true negative)	Healthy patient correctly predicted healthy	مريض سليم تنبأ النموذج بأنه سليم (صحيح)
TP (true positive)	Sick patient correctly detected	مريض فعلاً تم اكتشافه (صحيح)

Cell	Meaning (medical context)	المعنى (في السياق الطبي)
FP (false positive)	Healthy patient predicted sick: extra tests, anxiety, cost	سليم تنبأ بأنه مريض: فحوصات إضافية وقلق وتكلفة
FN (false negative)	Sick patient predicted healthy: MISSED DIAGNOSIS, no treatment	مريض تنبأ بأنه سليم: تشخيص فائت وبدون علاج (الأخطر)

EN The most costly error is the false negative (FN): a sick patient sent home untreated can die, whereas a false positive only leads to additional examinations.

AR الخطأ الأكثر كلفة هو السالب الكاذب (FN): المريض الذي لا يُكتشف مرضه يُترك دون علاج وقد تتدهور حالته، بينما الموجب الكاذب (FP) يؤدي فقط إلى فحوصات إضافية.

4. Regularization (L2, L1, Elastic Net)

4. التنظيم (L2 و L1 و Elastic Net)

Step 4.1 – L2 (Ridge) code analysis | الخطوة 4.1 – تحليل كود L2

```
def compute_cost_and_gradients_L2(X, y, w, b, lambda_):
    m = X.shape[0]
    cost, dw, db = compute_cost_and_gradients(X, y, w, b)
    cost = cost + (lambda_ / (2*m)) * np.sum(w**2)   # penalty term in the cost
    dw   = dw + (lambda_ / m) * w                    # extra term in dw
    # db unchanged: the bias b is NOT regularized
    return cost, dw, db
```

EN Line by line: (1) the base cross-entropy cost is computed; (2) the penalty (λ/2m)·Σw² punishes large weights; (3) its derivative (λ/m)·w is added to dw, so every update shrinks weights slightly (weight decay); (4) b is not penalized because it only shifts the decision boundary and does not control model complexity.

AR سطراً بسطر: (1) تُحسب كلفة الإنتروبيا المتقاطعة الأساسية؛ (2) حد العقوبة (λ/2m)·Σw² يعاقب الأوزان الكبيرة؛ (3) مشتقته (λ/m)·w يُضاف إلى dw فيتقلص كل وزن قليلاً عند كل تحديث (weight decay)؛ (4) لا نعاقب b لأنه يزيح حد القرار فقط ولا يتحكم في تعقيد النموذج.

```
w, b = initialize_parameters(X_train.shape[1])
w_l2, b_l2, costs_l2 = gradient_descent(X_train, y_train, w, b, alpha=0.5,
num_iters=2000,
                                        compute_fn=compute_cost_and_gradients_L2,
lambda_=1.0)

def evaluate(name, w_, b_, costs_):
    plt.figure(figsize=(6, 3.5)); plt.plot(costs_)
    plt.title(f"Cost convergence - {name}"); plt.xlabel("Iteration"); plt.ylabel("Cost")
    plt.grid(alpha=0.3); plt.show()
    yp = predict(X_test, w_, b_)
    print(name, "accuracy:", accuracy_score(y_test, yp))
    cm = confusion_matrix(y_test, yp); print(cm)
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues"); plt.title(name); plt.show()

evaluate("L2 (lambda=1.0)", w_l2, b_l2, costs_l2)
```

Step 4.2 – L1 (Lasso): complete the TODO | الخطوة 4.2 – L1: إكمال خلية TODO

EN The L1 penalty is (λ/m)·Σ|w| and its (sub)gradient is (λ/m)·sign(w). It pushes small weights to exactly zero, which performs feature selection.

AR عقوبة L1 هي (λ/m)·Σ|w| ومشتقتها الجزئية هي (λ/m)·sign(w). تدفع الأوزان الصغيرة إلى الصفر تماماً فتقوم باختيار الميزات (feature selection).

```
def compute_cost_and_gradients_L1(X, y, w, b, lambda_):
    m = X.shape[0]
    cost, dw, db = compute_cost_and_gradients(X, y, w, b)
    cost = cost + (lambda_ / m) * np.sum(np.abs(w))   # L1 penalty
    dw   = dw + (lambda_ / m) * np.sign(w)            # sub-gradient
    return cost, dw, db

w, b = initialize_parameters(X_train.shape[1])
w_l1, b_l1, costs_l1 = gradient_descent(X_train, y_train, w, b, alpha=0.5,
num_iters=2000,
                                        compute_fn=compute_cost_and_gradients_L1,
lambda_=0.1)
evaluate("L1 (lambda=0.1)", w_l1, b_l1, costs_l1)
```

Step 4.3 – Elastic Net | Elastic Net – 4.3 الخطوة

EN Elastic Net = ρ·L1 + (1−ρ)·L2. With ρ = 0.5 both penalties have equal influence.

AR Elastic Net = ρ·L1 + (1−ρ)·L2. عندما ρ = 0.5 يكون تأثير العقوبتين متساوياً.

```
def compute_cost_and_gradients_EN(X, y, w, b, lambda_, rho=0.5):
    m = X.shape[0]
    cost, dw, db = compute_cost_and_gradients(X, y, w, b)
    l1 = np.sum(np.abs(w))
    l2 = 0.5 * np.sum(w**2)
    cost = cost + (lambda_ / m) * (rho * l1 + (1 - rho) * l2)
    dw   = dw + (lambda_ / m) * (rho * np.sign(w) + (1 - rho) * w)
    return cost, dw, db

w, b = initialize_parameters(X_train.shape[1])
w_en, b_en, costs_en = gradient_descent(X_train, y_train, w, b, alpha=0.5,
num_iters=2000,
                                        compute_fn=compute_cost_and_gradients_EN,
lambda_=0.1)
evaluate("Elastic Net (lambda=0.1, rho=0.5)", w_en, b_en, costs_en)
```

Step 4.4 – Compare convergence and weights | الخطوة 4.4 – مقارنة التقارب والأوزان

```
# convergence curves on one figure
plt.figure(figsize=(7, 4))
plt.plot(costs0, label="No reg (a=0.1)")
plt.plot(costs_l2, label="L2"); plt.plot(costs_l1, label="L1"); plt.plot(costs_en,
label="Elastic Net")
plt.xlabel("Iteration"); plt.ylabel("Cost"); plt.legend(); plt.grid(alpha=0.3);
plt.show()

# weights comparison
W = pd.DataFrame({"No reg": w0.ravel(), "L2": w_l2.ravel(),
                  "L1": w_l1.ravel(), "ElasticNet": w_en.ravel()}, index=feature_names)
print(W.round(3))
W.plot(kind="bar", figsize=(12, 4)); plt.title("Weights per feature");
plt.grid(alpha=0.3); plt.show()
print("Weights equal to ~0 with L1:", (np.abs(w_l1.ravel()) < 1e-2).sum())

# bonus: effect of the decision threshold (question 3)
for t in [0.5, 0.4, 0.3]:
    yp = predict(X_test, w_l2, b_l2, threshold=t)
    tn, fp, fn, tp = confusion_matrix(y_test, yp).ravel()
    print(f"threshold={t}: FN={fn}  FP={fp}  recall={tp/(tp+fn):.2f}")
```

5. Answers to the Analysis Questions

5. أجوبة أسئلة التحليل

Q1. Why is normalization essential for gradient descent but optional for linear regression?

س1. لماذا يعد التطبيع ضرورياً للانحدار التدريجي واختيارياً في الانحدار الخطي؟

EN Gradient descent uses a single learning rate for all weights. If features have very different scales (age ≈ 50, cholesterol ≈ 250, oldpeak ≈ 1), the cost surface is elongated: one weight needs a tiny step, another a large one, so the algorithm oscillates, diverges (the exponential in the sigmoid overflows) or converges very slowly. Scaling to [0,1] makes the surface rounder and allows a larger α. Our linear regression with scikit-learn solves the problem in closed form (least squares), which does not depend on the scale, and with one feature scaling only changes the unit of the coefficient.

AR الانحدار التدريجي يستعمل معدل تعلم واحداً لكل الأوزان. إذا اختلفت مقاييس الميزات كثيراً (العمر ≈ 50، الكوليسترول ≈ 250، oldpeak ≈ 1) تصبح سطح الكلفة مستطيلاً: وزن يحتاج خطوة صغيرة جداً وآخر يحتاج خطوة كبيرة، فيتذبذب الخوارزم أو يتباعد (تفيض الدالة الأسية في السيغمويد) أو يتقارب ببطء شديد. التطبيع إلى [0,1] يجعل السطح أكثر استدارة ويسمح بقيمة α أكبر. أما الانحدار الخطي في scikit-learn فيُحل بصيغة مغلقة (المربعات الصغرى) لا تتأثر بالمقياس، ومع متغير واحد يتغير فقط تمييز وحدة المعامل.

Q2. Convergence with vs without regularization?

س2. مقارنة التقارب مع التنظيم وبدونه؟

EN Regularization does not make learning faster; it changes the objective. The regularized cost stays higher (it includes the penalty) and plateaus at a larger value, but the weights stay small, so training is more stable and less prone to overfitting. With L2 the curve is smooth and strictly decreasing; with L1 the sub-gradient sign(w) can cause small oscillations near zero, especially with a large α. In short: it stabilizes the weights, it does not accelerate convergence.

AR التنظيم لا يسرّع التعلم بل يغيّر دالة الهدف. تبقى الكلفة المنظَّمة أعلى (لأنها تتضمن العقوبة) وتستقر عند قيمة أكبر، لكن الأوزان تبقى صغيرة فيصبح التدريب أكثر استقراراً وأقل عرضة للإفراط في التعلم (overfitting). مع L2 يكون المنحنى ناعماً ومتناقصاً، ومع L1 قد تسبب المشتقة الجزئية sign(w) تذبذبات صغيرة قرب الصفر خاصة مع α كبيرة. باختصار: التنظيم يُثبّت الأوزان ولا يسرّع التقارب.

Q3. Most critical error and recommended threshold?

س3. أي خطأ هو الأخطر وما العتبة الموصى بها؟

EN The false negative (a sick patient classified as healthy) is the most critical, because the disease goes untreated. A false positive only leads to extra tests. I would therefore lower the threshold below 0.5, for example 0.3–0.4, to increase recall (sensitivity), accepting more false positives. The exact value is chosen on a validation set using the precision-recall or ROC curve, according to the cost of each error. (The bonus loop in step 4.4 shows the effect.)

AR السالب الكاذب (FN: مريض صُنّف سليماً) هو الأخطر لأن المرض يبقى دون علاج، بينما الموجب الكاذب يسبب فحوصات إضافية فقط. لذلك أوصي بخفض العتبة تحت 0.5، مثلاً 0.3–0.4، لرفع الاستدعاء (recall/sensitivity) مع قبول عدد أكبر من الموجبات الكاذبة. تُختار القيمة الدقيقة على مجموعة تحقق بالاعتماد على منحنى precision-recall أو ROC وحسب كلفة كل خطأ. (الحلقة الإضافية في الخطوة 4.4 توضح الأثر.)

Q4. What do the weights reveal? Do L1-selected variables match known risk factors?

س4. ماذا تكشف الأوزان؟ وهل تطابق متغيرات L1 عوامل الخطر المعروفة؟

EN Large positive weights increase the probability of disease, large negative weights decrease it. L2 shrinks all weights smoothly, while L1 sets the least useful ones to (almost) zero and keeps the strongest. On this dataset the variables that usually keep large weights are the number of major vessels coloured by fluoroscopy (ca), the type of chest pain (cp), thalassemia (thal), exercise-induced angina (exang), ST depression (oldpeak) and sex, which are consistent with clinically known indicators of coronary disease; maximum heart rate (thalach) usually has a negative weight (a high

achievable heart rate is protective). Check this against your own printed weights table. Remember that weights are only comparable because the features were scaled to the same range, and that correlation is not causation.

AR الأوزان الموجبة الكبيرة تزيد احتمال المرض والسالبة الكبيرة تقلله. L2 تقلّص كل الأوزان بنعومة، أما L1 فتجعل الأقل فائدة صفراً (أو قريباً منه) وتبقي الأقوى. في هذه البيانات عادة تحتفظ بأوزان كبيرة المتغيرات: عدد الأوعية الدموية الرئيسية (ca)، ونوع ألم الصدر (cp)، والثلاسيميا (thal)، والذبحة الناتجة عن الجهد (exang)، وانخفاض ST (oldpeak)، والجنس، وهي متوافقة مع المؤشرات السريرية المعروفة لمرض الشريان التاجي؛ أما أقصى معدل لضربات القلب (thalach) فوزنه عادة سالب (معدل أعلى يعني حماية أكبر). تحقق من ذلك في جدول الأوزان الذي طبعته أنت. وتذكّر أن الأوزان قابلة للمقارنة فقط لأن الميزات قيست على نفس المجال، وأن الارتباط لا يعني السببية.

Q5. Ridge, Lasso or Elastic Net: when?

س5. متى نفضّل Ridge أو Lasso أو Elastic Net؟

EN See the summary table below.

AR انظر الجدول الملخص أدناه.

Method	Prefer it when…	Effect	Arabic / الاستخدام
Ridge (L2)	Many correlated features, all of them useful	Shrinks weights, none exactly 0	ميزات كثيرة مترابطة وكلها مفيدة؛ تقلّص الأوزان دون حذفها
Lasso (L1)	Many features, only a few relevant; need interpretability	Sparse model, feature selection	ميزات كثيرة قليلها مهم؛ نريد نموذجاً مبسطاً؛ يحذف الميزات
Elastic Net	Many correlated features AND need selection (p > n)	Compromise: sparsity + stability	ميزات مترابطة مع الحاجة للاختيار؛ حل وسط بين الاثنين

6. Final Checklist

6. قائمة التحقق النهائية

- Environment tested and notebook TP1_Regression.ipynb created
- Part 2 (a→h): table of Ridge/Lasso/ElasticNet filled with your results
- Part 3 (a→c): cost curve decreasing, accuracy and confusion matrix interpreted
- Part 4: L2, L1 and Elastic Net trained with the given α, iterations and λ
- Part 5: the five questions answered in your own words

- تم اختبار البيئة وإنشاء الدفتر TP1_Regression.ipynb
- الجزء 2 (من a إلى h): جدول Ridge/Lasso/ElasticNet مملوء بنتائجك
- الجزء 3 (من a إلى c): منحنى الكلفة متناقص، ومعدل الدقة ومصفوفة الالتباس مفسَّران
- الجزء 4: تدريب L2 و L1 و Elastic Net بقيم α والتكرارات و λ المطلوبة
- الجزء 5: الإجابة على الأسئلة الخمسة بأسلوبك الخاص